A Brief Exploration and Implementation of the CART Algorithm for Classification Tasks

Jonah Catlin

For this project, I chose to adopt a **Decision Tree** algorithm, more specifically the **CART (Classification and Regression Tree)** algorithm, implemented for a classification task. I will begin with a brief overview of what this algorithm is and what it is used for. Second, I will provide a detailed mathematical explanation of how the algorithm works. Next, I will discuss my python implementation of the algorithms. Finally, I will showcase the algorithms in action on two datasets, and explain how I tested their performance.

Algorithm Overview

A Decision Tree algorithm is a classification algorithm which works in a generic sense by iteratively splitting a dataset across hyperplanes in the feature space, in a manner which minimizes some measure of "impurity." This process creates a set of nodes (subsets of the original dataset) and connections which form a tree structure (hence the name). The algorithm continues to split nodes until it encounters one of the following conditions:

- A node is fully homogeneous ("impurity" = 0)
- All feature values in a node are identical
- A node is too small to split properly
- A node is too deep to be split further

In the case one of these conditions is met, a node will not be split. Instead, the node will be assigned a “value” based on the classes of its points, and marked as a “leaf” node. Once all nodes have either been split or marked as leaf nodes, the decision tree has been created, and the algorithm terminates.

To classify a new point with a decision tree, you start at the root of the tree structure, and trace a path down the tree by checking where the point lies with respect to the hyperplane boundaries. Once you reach a leaf node, the point is classified into the group corresponding with the value of this leaf node.

Mathematical Description

With this brief overview covered, let’s lay the groundwork for the detailed description of the CART algorithm with some notation. Since we are addressing a classification task, each point comes with a set of feature values, $X_i = [x_{i1}, \ldots, x_{id}]$, and a label, $y_i \in \{1, \ldots, K\}$, where $i \in \{1, \ldots, n\}$ is the index of the point. Here, we let d be the dimension of the feature space, and K be the number of classes.

Datasets often contain both numerical and categorical features. To account for this possibility, we will let $x_{ij} \in \mathbb{R}$ if feature j is numerical, and let $x_{ij} \in L_j$ if feature j is categorical, for $j \in \{1, \ldots, d\}$. In the latter case, we let L_j denote the finite set of categorical levels for feature j, with $\left|L_j\right|$ being the number of levels in this set.

Next, to facilitate easy splitting of the data, we will define a collection $\{C_m\}$ of points to be a subset of the whole dataset $\{(X_i, y_i)\}_{i=1}^{n}$, $m \in \mathbb{N}$. Here, we let m denote an index by which to uniquely identify a collection. Additionally, each collection $\{C_m\}$ will have a corresponding depth, D_m and number of points $|\{C_m\}|$. With these foundations laid, we now look to the "impurity" function.

For the CART algorithm, the choice of "impurity" function used is the Gini index. The Gini index can be formulated as $Gini\ index = 1 - \sum_{k=1}^{K} p_k^2$, with p_k representing the proportion of elements in a set falling into group k. The Gini index ranges in value from 0 to 1, with a value of 0 representing complete homogeneity, while a value of 1 represents maximal dissimilarity between elements. To allow for simple application of this measure, I will define $p_k(\{C_m\})$ to be the proportion of points in a collection $\{C_m\}$ where $y_i = k$, for $k \in \{1, \ldots, K\}$.

It should be noted that splitting a set results in multiple subsets. As a result, our impurity measure will need to be defined as an average of the Gini indices of the subsets, rather than that of the original set. In addition, the difference between splitting across a categorical and numerical feature necessitates a separate function for each.

When feature j is categorical, we can simply take the average of the Gini indices across each categorical level. Defining $\{C_{m,l}\}$ as the subset of $\{C_m\}$ where $x_{ij} = l$, we get the following formulation: $c_j(\{C_m\}) := \frac{1}{|L_j|} \sum_{l \in L_j} \left(1 - \sum_{k=1}^{K} \left(p_k(\{C_{m,l}\})\right)^2\right)$.

When feature j is numerical, it gets more complicated. Since there are not predetermined lines to split the data across, we will instead define a boundary $b \in (B_0, B_1]$, where $B_0 = min\left[x_{ij} \in \{C_m\}\right]$ and $B_1 = max\left[x_{ij} \in \{C_m\}\right]$. The endpoints of this interval are defined in this manner to ensure that there will (almost) always be points on both sides of the boundary (The only exception is if all points in node $\{C_m\}$ share a value for feature j). We can then define $\{C_{m,b}\}$ as the subset of $\{C_m\}$ where $x_{ij} \geq b$. The subset of $\{C_m\}$ where $x_{ij} < b$ can then be represented as $\{C_m\} \backslash \{C_{m,b}\}$. With these two subsets established, we can minimize their average Gini index across all choices of b to obtain the following formulation:

$$n_j\left(\{C_m\}\right) := min_b\left[1 - \frac{1}{2}\sum_{k=1}^{K}\left(\left(p_k\left(\{C_{m,b}\}\right)\right)^2 + \left(p_k\left(\{C_m\} \backslash \{C_{m,b}\}\right)\right)^2\right)\right]$$

With a formula for the Gini index of a split of collection $\{C_m\}$ across feature j for both categorical and numerical features, we can compile them into one function:

$g_j\left(\{C_m\}\right) := c_j\left(\{C_m\}\right)$ if feature j is categorical, $\quad g_j\left(\{C_m\}\right) := n_j\left(\{C_m\}\right)$ if feature j is numerical.

As an additional side note, if all points in collection $\{C_m\}$ share a value for feature j, it could lead to an infinite loop. Thus, in this case, we will let $g_j\left(\{C_m\}\right) = 1$ to prevent a split, as a split would create one child node with an identical collection of points.

Now that we have an impurity function which is applicable for all features, we can use it to choose which feature is best to split across, and split across it. We can accomplish this by minimizing the function across our features $j \in \{1, \ldots, d\}$, and picking the minimizer. Thus, we

define the split feature, S, as $S := argmin_j\left[g_j\left(\{C_m\}\right)\right]$. Next, to split the collection $\{C_m\}$ across the feature S, we will recall the subgroups used to

To actually split the node $\{C_m\}$ across feature S, we will need to recall the subgroups used earlier to calculate the Gini index across a split. For a split across a categorical feature j, we used $\left\{\{C_{m,l}\}\right\}_{l\in L_j}$. This expression can be easily modified to find the subgroups for an optimal categorical split, $\left\{\{C_{m,l}\}\right\}_{l\in L_S}$. A split across a numerical feature is once again more complicated. We used subgroups $\left\{\{C_{m,b}\}, \{C_m\}\backslash\{C_{m,b}\}\right\}$ previously, but this does not specify the optimal split point. Thus, we define $b^* = argmin_b\left[1 - \frac{1}{2}\sum_{k=1}^{K}\left(\left(p_k\left(\{C_{m,b}\}\right)\right)^2 + \left(p_k\left(\{C_m\}\backslash\{C_{m,b}\}\right)\right)^2\right)\right]$ as the optimal boundary for an optimal numerical split, and let the optimal subgroups be $\left\{\{C_{m,b^*}\}, \{C_m\}\backslash\{C_{m,b^*}\}\right\}$. Once these optimal subgroups are identified, we create new nodes with collections $\{C_{m'}\}$, using appropriate indices m' for each.

Lastly, as mentioned above, the algorithm must have every node split or set as a leaf node before terminating. To allow for this behavior, the algorithm will check the four aforementioned conditions, and stop if any one is met:

- A node is fully homogeneous (“impurity” = 0):
    - $1 - \sum_{k\in K}\left(p_k\left(\{C_m\}\right)\right)^2 = 0$
- All feature values in a node are identical
    - $\exists a \in \mathbb{R}^d \; s.t. \; X = a \; \forall X \in \{C_m\}$

- A node is too small to split properly
    - $\left|\{C_m\}\right| < Minimum\ Size$
- A node is too deep to be split further
    - $d_m = Maximum\ Depth$

Here, the $Minimum\ Size$ and $Maximum\ Depth$ are hyperparameters of the model, and each can take on any positive integer value.

In the case that any of these conditions are met, the node is assigned a value, $V\left(\{C_m\}\right) = argmax_k\left(p_k\left(\{C_m\}\right)\right)$, which is the most common class label out of all of the points in its collection.

With all of the formulas set out, all that is left is to initialize the algorithm and let it run to create the decision tree. We start the algorithm by creating a root node with dataset $\{C_0\}$ and a depth of $d_0 = 0$. The algorithm will then go into a loop and repeat the following until termination:

- Check if current node $\{C_m\}$ can be split
    - If it can, calculate a split and a new set of nodes as described above, and then assign these nodes to $\left\{\{C_M\},\ \{C_{M+1}\},\ \ldots\right\}$, with M being the next available integer index
        - Additionally, let the corresponding $\{d_M,\ d_{M+1},\ \ldots\}$ each equal $d_m + 1$
    - If it cannot, set the value of the node and mark it as a leaf node
- Check if node $\{C_{m+1}\}$ exists

- If it exists, set $\{C_{m+1}\}$ as the current node and proceed to beginning of next iteration
- If it does not exist, terminate the algorithm

With this “psuedo-code” laid out, we can now discuss some details about the implementation.

Python Implementation Process and Cut Content

The bulk of my time on this project was spent implementing the aforementioned CART algorithm, but the majority of the code was simply that, implementation of what was described above. As a result, I will focus this section on the interesting bits and pieces that went into the development process. I will also briefly discuss another piece which I do not have enough room to explore fully in this report.

The implementation of this algorithm was an interesting learning experience for me, as I had to take up a wide range of tools which I had never touched before. A decision tree can be boiled down to a network of nodes and connections. Considering this modular structure, I figured the best approach was an object-oriented approach. This was a perfectly good approach, with the one caveat that I had never made anything with object-oriented programming before. As you would expect, teaching myself how to code and use classes from scratch was quite the experience. I am certain that I have violated numerous OOP principles in my 900-something line long quest of a python file (consisting mostly of spaces and comments), but I have a functional algorithm, which is more than good enough for a first attempt. I will include this python file and an accompanying python notebook in my project submission, so feel free to explore it as much as you would like.

Next, before we get to the application to actual datasets, it is important to bring up what had to be left out of the detailed analysis. In addition to implementing the CART algorithm in python, I also coded a Random Decision Forest classifier. For a quick explanation, it creates several smaller decision trees with CART, with each tree being trained on a smaller portion of the dataset. Each of the smaller datasets features a set of a specified number points and labels sampled with replacement from a small subset of the original data. Additionally, each smaller dataset has a subset of the original features (chosen without replacement) which it is sensitive to, to prevent trees from being overly correlated. The proportion of points and features used in this sampling process become hyperparameters for the model, along with the number of trees in the forest. It is important to note that there are two separate testing functions which were built to evaluate the algorithms, $tts()$ and $multi_tts()$. The former is for a single decision tree, while the latter is designed for a random forest. I bring this up to highlight the fact that testing a single decision tree using $multi_tts()$ with appropriate hyperparameter choices will result in unusual behavior, due to the way the data points are sampled.

Weather Dataset

An interesting toy dataset I have seen used to help show the process behind a decision tree is the weather dataset. It is a small dataset containing a set of categorical variables describing various weather conditions, and a set of labels (“Decisions”) describing whether the conditions are correct to go and play. Pictured to the right is a screenshot of this dataset, as a pandas df.

	Outlook	Temperature	Humidity	Wind	Decision
0	sunny	hot	high	weak	No
1	sunny	hot	high	strong	No
2	overcast	hot	high	weak	Yes
3	rainfall	mild	high	weak	Yes
4	rainfall	cool	normal	weak	Yes
5	rainfall	cool	normal	strong	No
6	overcast	cool	normal	strong	Yes
7	sunny	mild	high	weak	No
8	sunny	cool	normal	weak	Yes
9	rainfall	mild	normal	weak	Yes
10	sunny	mild	normal	strong	Yes
11	overcast	mild	high	strong	Yes
12	overcast	hot	normal	weak	Yes
13	rainfall	mild	high	strong	No

To help demonstrate the process, I will walk through a few steps of the algorithm result when applied to this dataset. Pictured right is the output for the first node. The data is identical to that listed above, as this is the root node. Below the data, you can see that the algorithm has chosen to split across the three levels of the "Outlook" feature. Thus, the algorithm will create three new nodes, with one for each of the levels of this feature. Next, here are the outputs for the three newly created nodes:

```
node id: 0

     Outlook Temperature Humidity    Wind Decision
0      sunny         hot     high    weak       No
1      sunny         hot     high  strong       No
2   overcast         hot     high    weak      Yes
3   rainfall        mild     high    weak      Yes
4   rainfall        cool   normal    weak      Yes
5   rainfall        cool   normal  strong       No
6   overcast        cool   normal  strong      Yes
7      sunny        mild     high    weak       No
8      sunny        cool   normal    weak      Yes
9   rainfall        mild   normal    weak      Yes
10     sunny        mild   normal  strong      Yes
11  overcast        mild     high  strong      Yes
12  overcast         hot   normal    weak      Yes
13  rainfall        mild     high  strong       No

New Groups: ['Outlook = sunny', 'Outlook = overcast', 'Outlook = rainfall']
```

The first of these new nodes was created with the data where "Outlook" = "sunny". The algorithm decided to split this node further across the "Humidity" feature. This choice makes sense, as each level of "Humidity" has exactly one type of "Decision".

```
node id: 1

   Outlook Temperature Humidity    Wind Decision
0    sunny         hot     high    weak       No
1    sunny         hot     high  strong       No
7    sunny        mild     high    weak       No
8    sunny        cool   normal    weak      Yes
10   sunny        mild   normal  strong      Yes

New Groups: ['Humidity = high', 'Humidity = normal']
```

The second new node is that with the level "Outlook" = "overcast". This one is interesting, as the algorithm has not given us another feature to split across. If we look at the labels ("Decisions"), we can see that all of them are "Yes", which means that the data is homogeneous and thus has an impurity of 0. Thus, we can conclude that the algorithm

```
node id: 2

     Outlook Temperature Humidity    Wind Decision
2   overcast         hot     high    weak      Yes
6   overcast        cool   normal  strong      Yes
11  overcast        mild     high  strong      Yes
12  overcast         hot   normal    weak      Yes

New Groups: []
```

found this node to be a leaf node with value “Yes”, thus deciding not to split any further.

The final new node from this iteration is the one with “Outlook” = “rainfall”. The algorithm decided that the best split here is to split across the “Wind” feature. This yet again makes sense, as each level of “Wind” has exactly one type of “Decision” in this case.

```
node id: 3

     Outlook Temperature Humidity    Wind Decision
3   rainfall        mild     high    weak      Yes
4   rainfall        cool   normal    weak      Yes
5   rainfall        cool   normal  strong       No
9   rainfall        mild   normal    weak      Yes
13  rainfall        mild     high  strong       No

New Groups: ['Wind = weak', 'Wind = strong']
```

After this first iteration, one of the nodes (2) has been classified as a leaf node. The other two can be split here, but will be split into leaf nodes at the next iteration, due to their having one type of “Decision” for each level of the relevant split. Thus, with an insight into how the algorithm works on actual data and reasonable proof that it works in some capacity, I will conclude the demonstration for this dataset here and move on to the next dataset. If you wish to see the remainder of the output, you can find it in the aforementioned python notebook.

Iris Dataset

The next data I wanted to test the algorithm on was the iris dataset. This choice made sense, as it would allow me to test the algorithm on a larger (in comparison) dataset, with numerical features.

To test the performance of the algorithm, I wrote a train-test-split function, $tts()$. In short, this function takes in a dataset, integers for how many points to train and test with, and the hyperparameters to build a decision tree with. For a dataset of this size, overfitting from the tree running too deep would be relatively unlikely, so I have made the hyperparameters fairly generous. The function also takes in a seed value, to allow for consistent random choices in the

training/testing data. The function then simply trains a tree off of the training data, and tests it on each set. It then prints a short message and returns an accuracy score for each subset of the data. Below is a screenshot of an output for one of these trees: (Note that "df" is the dataset, and the list is a list of feature types)

```
DT.Tree.tts(120, 30, df, ['num', 'num', 'num', 'num'], tts_Seed = 0)
```

✓ 0.2s

```
Training size: 120
Testing size: 30

Train Accuracy: 0.975
Test Accuracy: 0.9666666666666667
```

Here, using 80% of the data for training, we are able to get a 96.67% testing accuracy on this seed. Despite this, I do not fully trust the accuracy of this accuracy measure, as it is heavily dependent on the choice of seed, and has increments of 3.33% per data point due to the small number of data points. To get a more realistic measure, I ran this $tts()$ function 100 times with 100 different seeds, and took the average of the accuracy values for each of these runs. Here is the output: (Note that "printing = False" simply prevents a print statement from occurring for every instance of the $tts()$ function)

```
avg_Train_Acc = 0
avg_Test_Acc = 0

for seed in range(100):

    acc = DT.Tree.tts(120, 30, df, ['num', 'num', 'num', 'num'], tts_Seed = seed, printing = False)

    print(f'Building Tree {seed + 1}/100', end = '\r')

    avg_Train_Acc += 0.01 * acc[0]
    avg_Test_Acc += 0.01 * acc[1]

print(f'Average training accuracy: {avg_Train_Acc}')
print(f'Average testing accuracy: {avg_Test_Acc}')
```

✓ 58.2s

```
Average training accuracy: 0.9852500000000014
Average testing accuracy: 0.9406666666666675
```

After 100 runs, we have an average training accuracy of **98.5%** and an average testing accuracy of **94.1%**. Although it is not as good as the prior result, it is still a pretty solid accuracy for an algorithm this simple, and I will take that as a win. With this victory, I will conclude this brief exploration into using the CART algorithm for classification. If you want to experiment with this implementation of the algorithm, you can also find this example in the aforementioned python notebook. Do, though, be aware that doing 100 runs could take a while.